

EDA(Exploratory Data Analysis)

```
[ ]: EDA(Exploratory Data Analysis)is the process of examining ,summarizing and visualising data before building machine learning models.
```

```
[ ]: # Why EDA is important?
1.Understand the structure of the dataset
2.Identify missing values
3.Detect outliers
4.undestand feature distributions
5.Find relationships between variables
6.Check data quality issues
7.prepare data for machine learning
```

▼ Types of EDA

```
[ ]: #Analyzing one variable at a time.

#numerical Features
Mean,median,Mode,Standard Deviation ,variance,Histogram,Box Plot

#Categorical Features
Value counts,Frequency Distribution,Bar Chart
```

```
[5]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")

print(df.head())
```

```
print(df.head())
print(df.info())
print(df.columns)
```

```
   total_bill  tip  sex smoker  day  time  size
0      16.99  1.01 Female    No  Sun  Dinner    2
1      10.34  1.66   Male    No  Sun  Dinner    3
2      21.01  3.50   Male    No  Sun  Dinner    3
3      23.68  3.31   Male    No  Sun  Dinner    2
4      24.59  3.61 Female    No  Sun  Dinner    4
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 244 entries, 0 to 243
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	total_bill	244 non-null	float64
1	tip	244 non-null	float64
2	sex	244 non-null	category
3	smoker	244 non-null	category
4	day	244 non-null	category
5	time	244 non-null	category
6	size	244 non-null	int64

```
dtypes: category(4), float64(2), int64(1)
```

```
memory usage: 7.4 KB
```

```
None
```

```
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'], dtype='object')
```

```
[2]: #1. Descriptive Statistics
print(df['total_bill'].describe())
```

```
count    244.000000
mean      19.785943
std        8.902412
min        3.070000
25%       13.347500
50%       17.795000
```

```
[2]: #1. Descriptive Statistics
print(df['total_bill'].describe())

count    244.000000
mean      19.785943
std        8.902412
min        3.070000
25%       13.347500
50%       17.795000
75%       24.127500
max       50.810000
Name: total_bill, dtype: float64
```

2.Histogram

- A Histogram shows the distribution of a numerical features
- It divides data into intervals(bins) and shows how many observations fall into each interval.

```
sns.histplot(df['tip'],bins=20,kde=True) plt.title("distribution of Tips") plt.show()
```

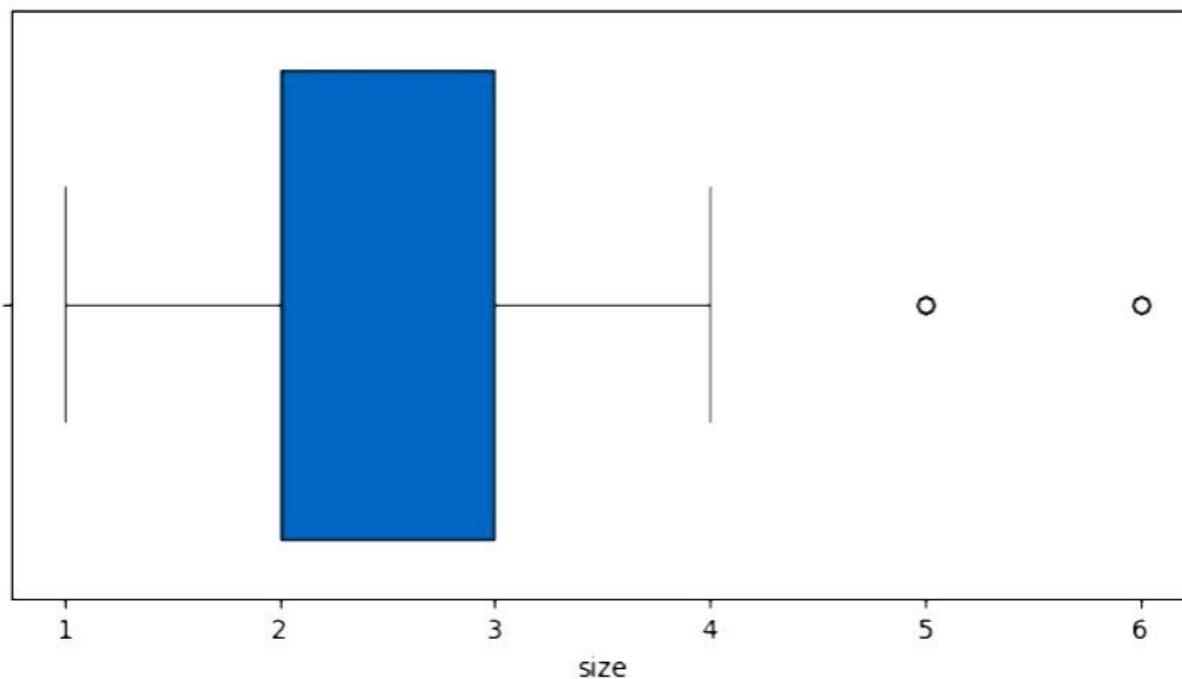
3. Box plot

```
[6]: plt.figure(figsize=(8,4))
sns.boxplot(x=df['size'])
plt.title("Box PLOT of Size")
plt.show()
```

Box PLOT of Size



Box Plot of Size



2. Categorical Variable: Day

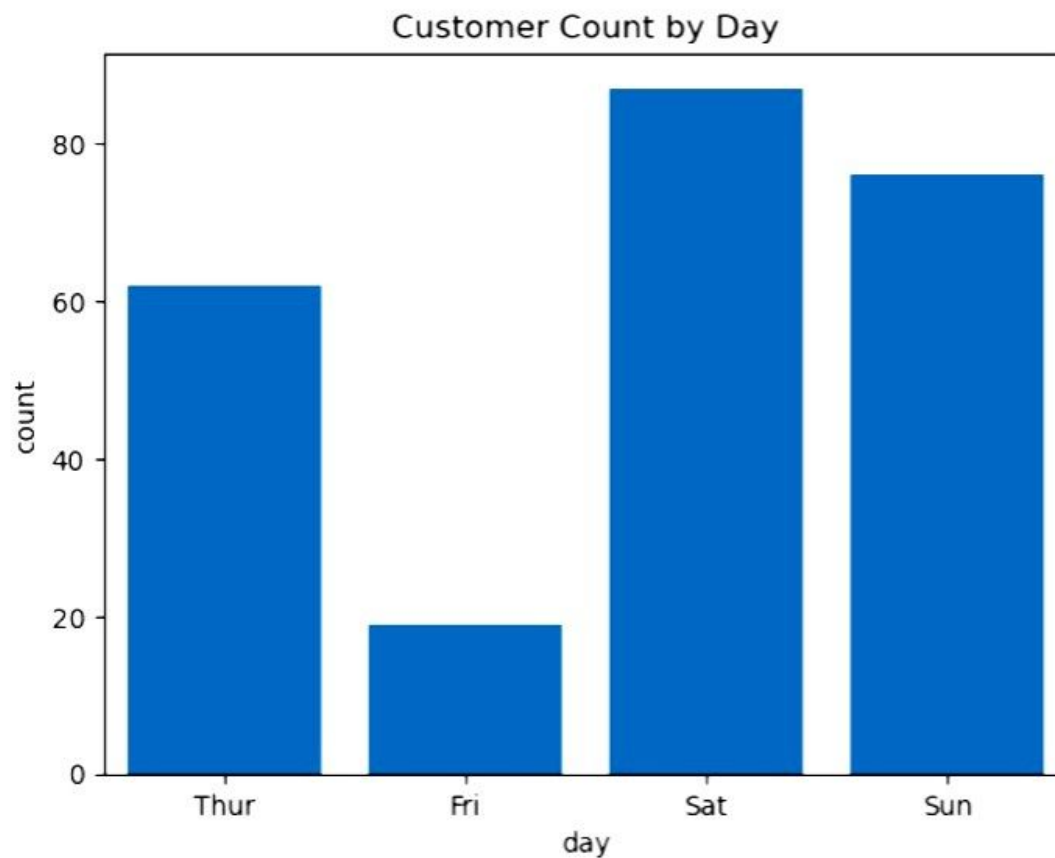
Value counts

```
print(df['day'].value_counts())
```

```
day
Sat      87
Sun      76
Thur     62
Fri      19
Name: count, dtype: int64
```

Name: count, dtype: int64

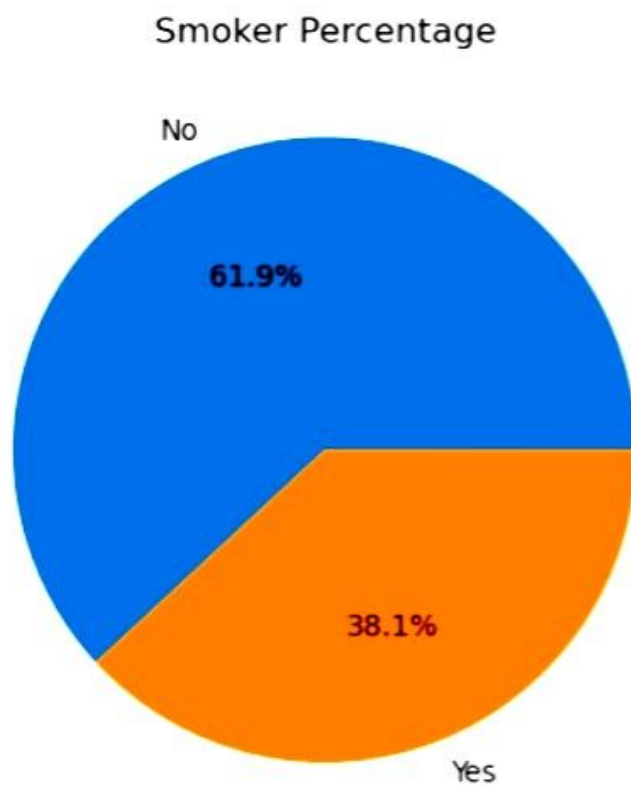
```
31: ## Count Plot
sns.countplot(x='day',data=df)
plt.title("Customer Count by Day")
plt.show()
```



```
32: # Pie chart
df['smoker'].value_counts().plot()
```

day

```
# Pie chart
df['smoker'].value_counts().plot(
    kind = 'pie',
    autopct = '%1.1f%%'
)
plt.title("Smoker Percentage") # to find hw many s and no person
plt.ylabel("")
plt.show()
```



```
[10]: ##Outlier Detection using IQR
Q1 = df['total_bill'].quantile(0.25)
Q3 = df['total_bill'].quantile(0.75)

IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[
    (df['total_bill'] < lower) | (df['total_bill'] > upper)]
print("Outliers:", len(outliers))
```

Outliers: 9

Bivariant Analysis

- Bivariant Analysis means analyzing two variables together to understand the relationships between them. Bi = Two Variant = Variables
- Example:
 1. Study hours vs Exam score
 2. Total bill vs fare

```
[ ]: # Types of Bivariant Analysis
```

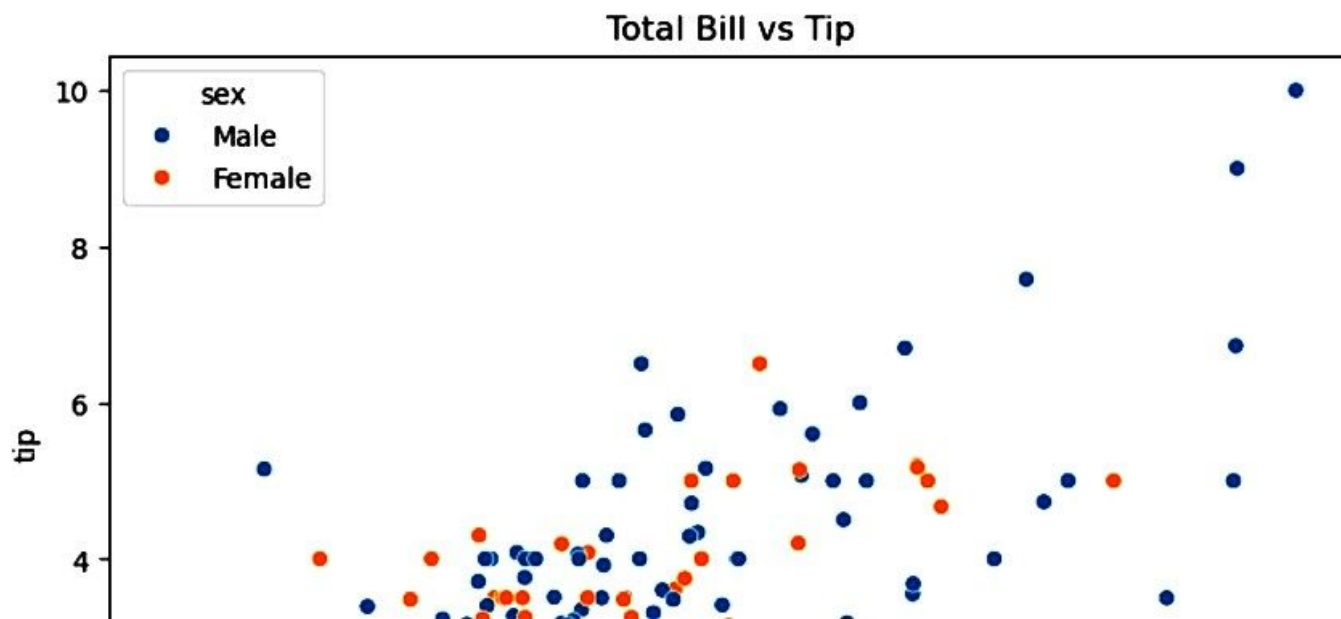
variable1	variable2	visualization
Numerical	Numerical	Scatter Plot
Categorical	Numerical	Bar chart
Categorical	Categorical	Grouped statistics / Crosstab

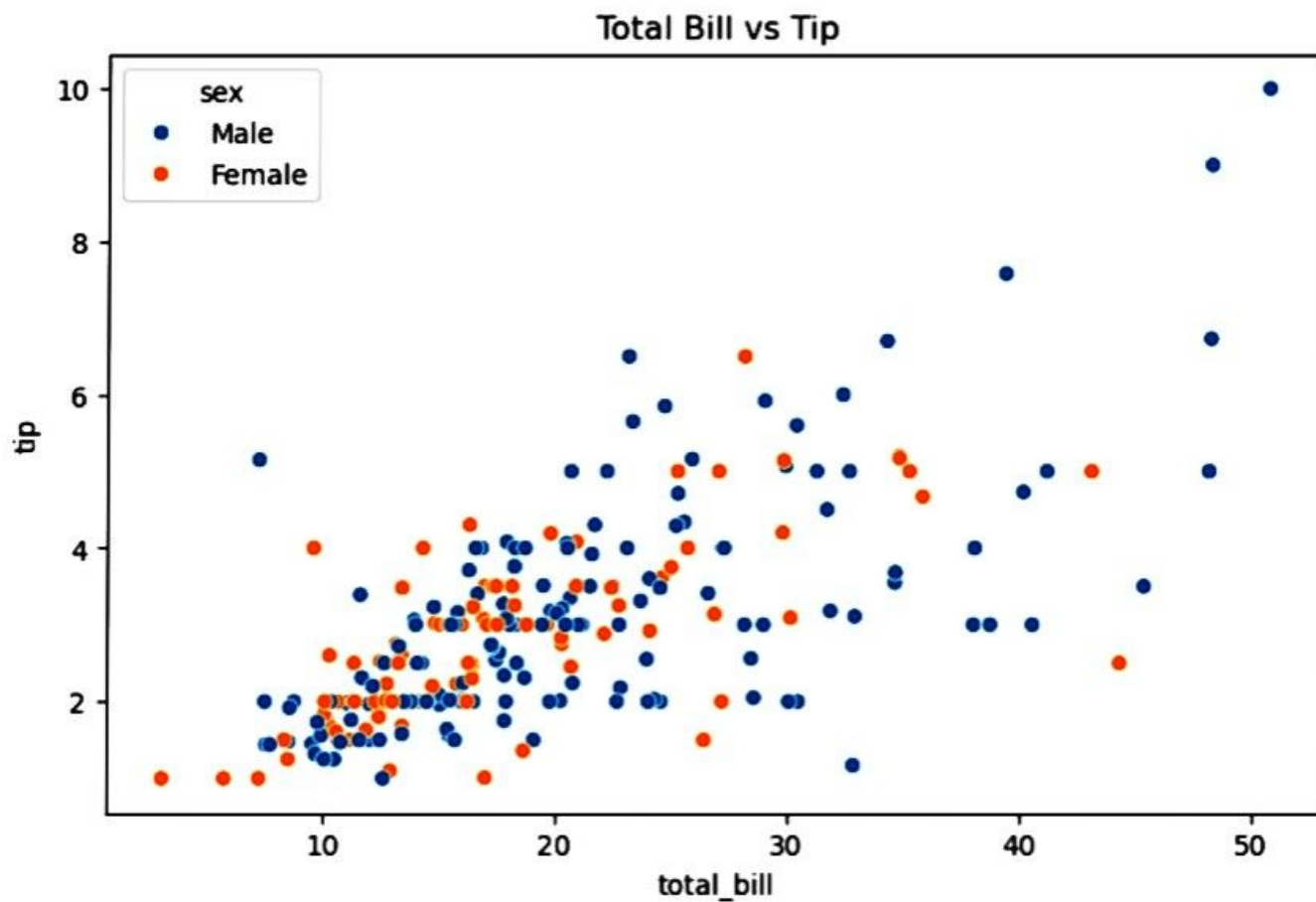
Numerical vs Numerical

Used to determine whether one numerical variable affects another.

Total Bill vs Tip

```
[12]: plt.figure(figsize=(8,5))
sns.scatterplot(
    x='total_bill',
    y='tip',
    hue='sex',
    data=df
)
plt.title("Total Bill vs Tip")
plt.show()
```

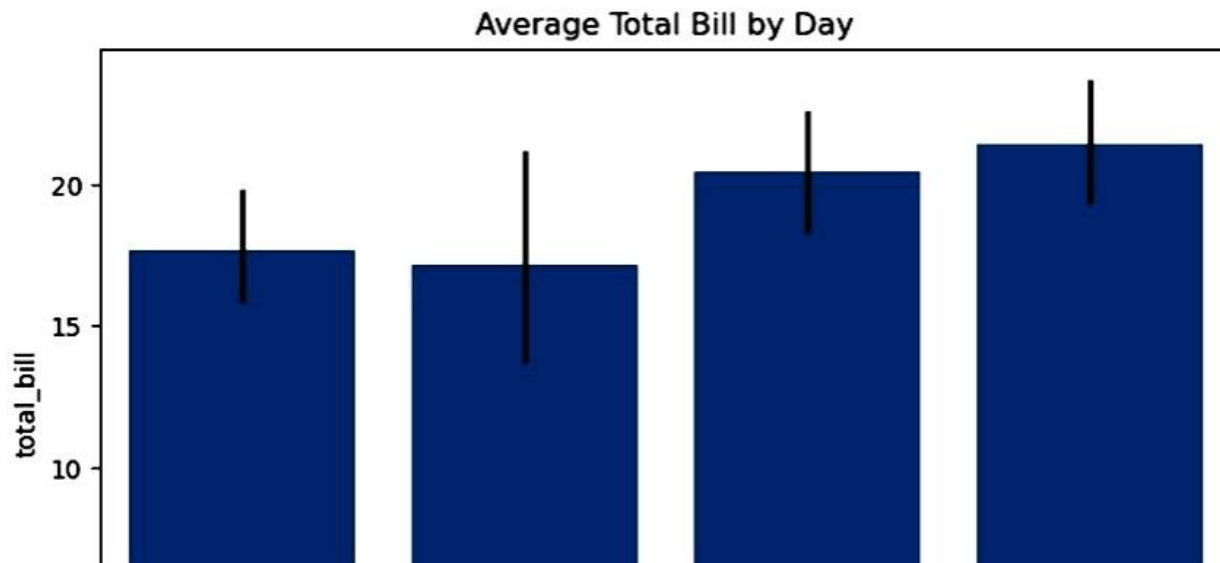




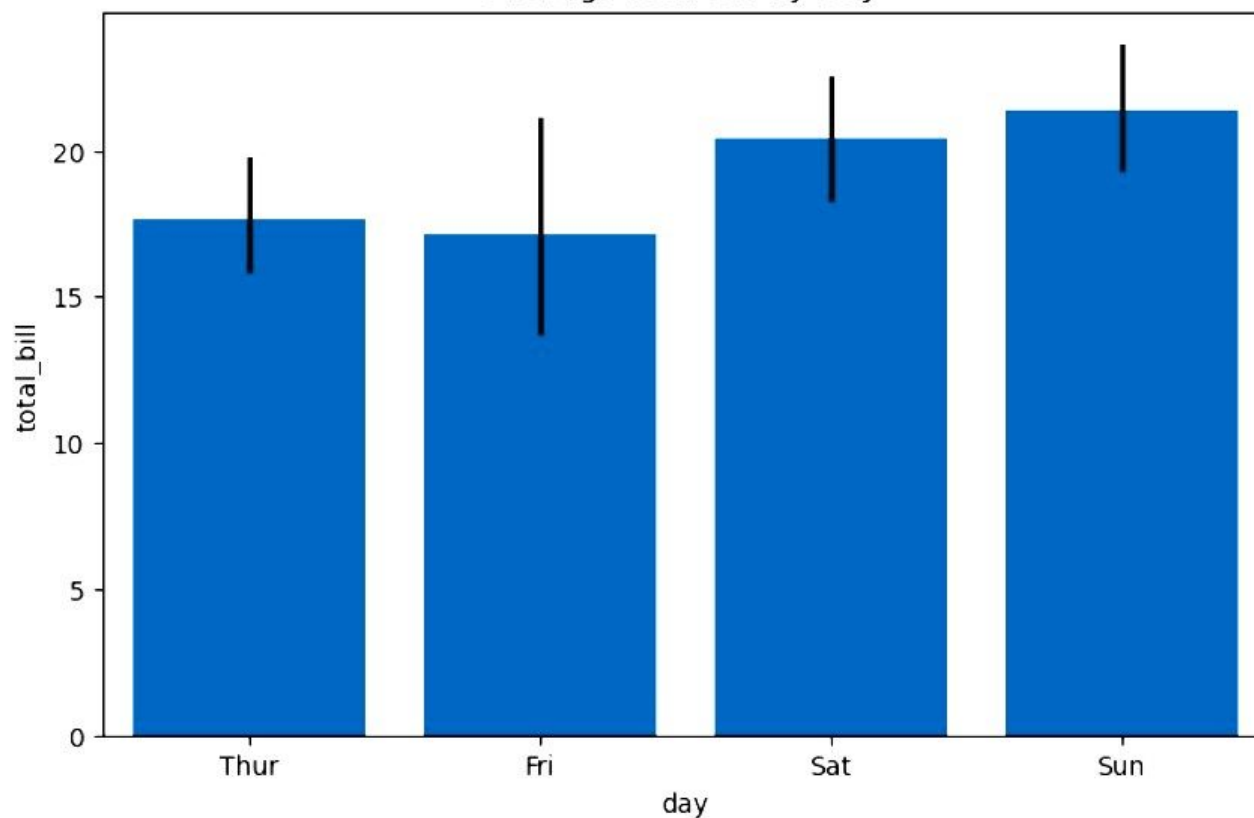
Categorical vs Numerical

Day vs total Bill Compare average values across categories

```
[14]: # Bar chart
plt.figure(figsize=(8,5))
sns.barplot(
    x='day',
    y='total_bill',
    data=df
)
plt.title("Average Total Bill by Day")
plt.show()
```



Average Total Bill by Day



```
[45]: #Grouped Statistics
#Mean Bill by Day
print(
    df.groupby('day', observed=False)['total_bill'].mean() # if we give true means warning will come
)

#Multiple Statistics
print(
```

```
[15]: #Grouped Statistics
#Mean Bill by Day
print(
    df.groupby('day',observed=False)['total_bill'].mean()    # if we give true means warning will come
)

#Multiple Statistics
print(
    df.groupby('day',observed=False)['total_bill']
    .agg(['mean','median','min','max'])
)
```

```
day
Thur    17.682742
Fri     17.151579
Sat     20.441379
Sun     21.410000
Name: total_bill, dtype: float64
      mean  median   min   max
day
Thur  17.682742   16.20  7.51  43.11
Fri   17.151579   15.38  5.75  40.17
Sat   20.441379   18.24  3.07  50.81
Sun   21.410000   19.63  7.25  48.17
```

▼ Sex vs Smoker

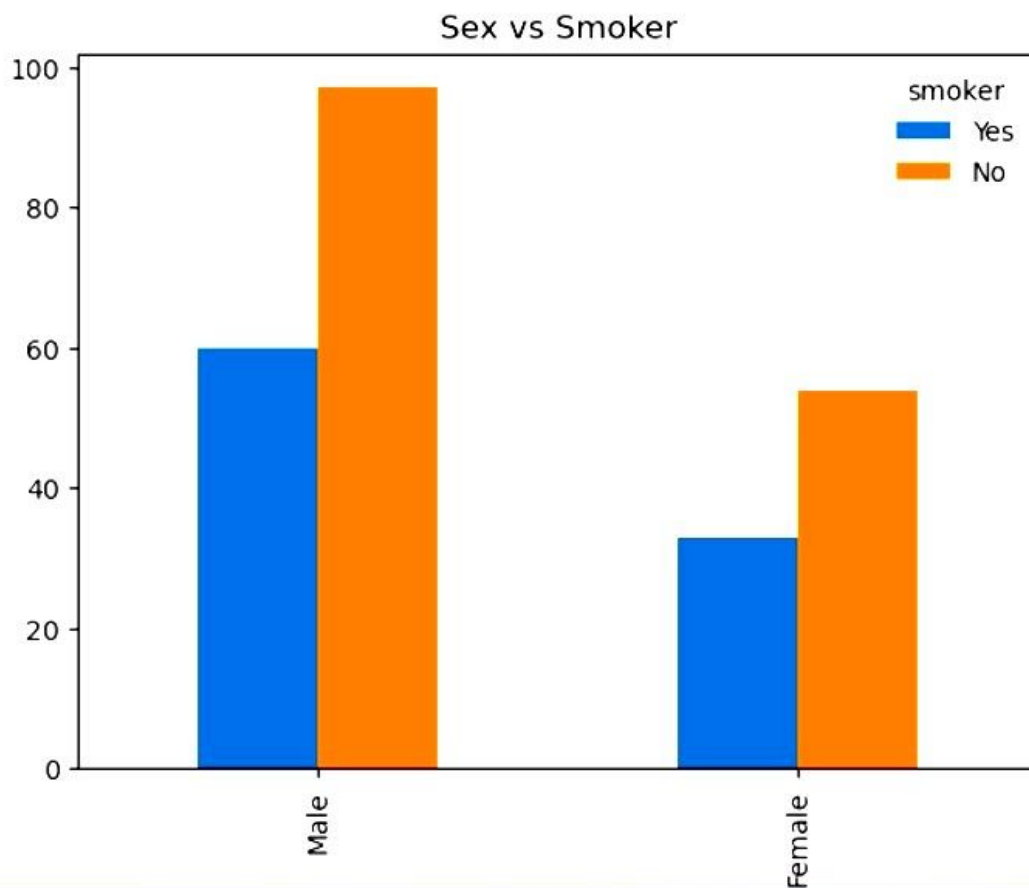
Compare frequencies between categories.

Crosstab

It is used to analyze the relationships between two categorical variables

[16]:

```
#Bar chart
pd.crosstab(
    df['sex'],
    df['smoker']
).plot(kind='bar')
plt.title("Sex vs Smoker")
plt.show()
```



Multivariate Analysis

- Multivariate Analysis means analyzing three or more variables simultaneously to understand complex relationships and patterns in data.

Example:

Instead of studying:

1. Height alone(Univariate)
2. Height vs Weight(Bivariate)

We Study:

Height vs Weight vs Age vs Gender This is Multivariate Analysis.

Objectives of Multivariate Analysis

1. Discover hidden relationships
2. Understand interactions among multiple features
2. Detect clusters and patterns
3. Support feature selection
5. Prepare data for Machine learning

1.Pairplot

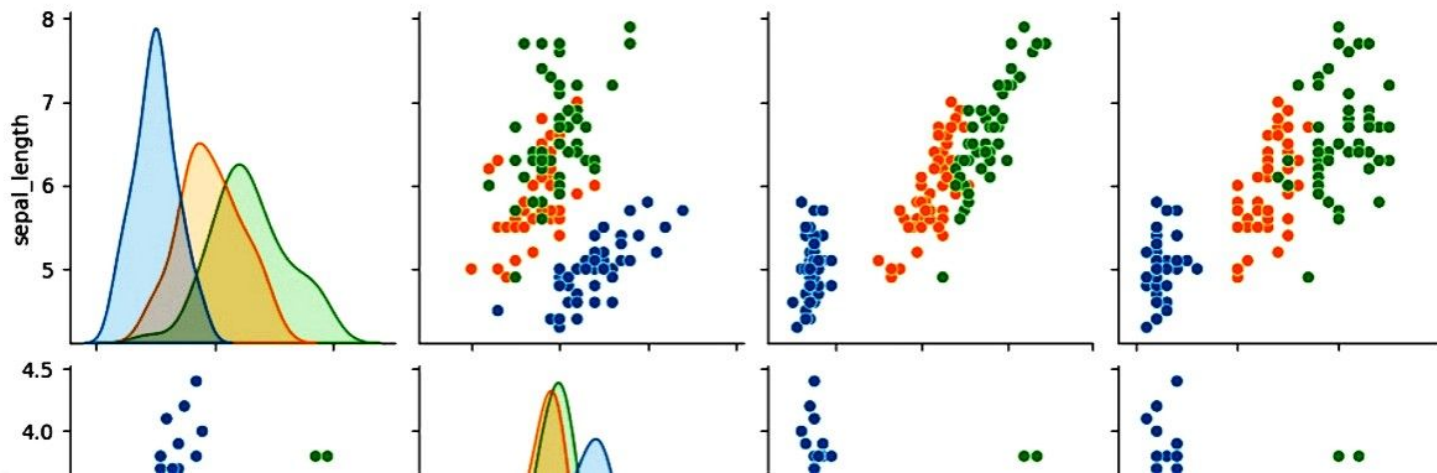
- A pairplot compares every numerical feature with every other numerical feature.
- It automatically creates:
 - 1.Scatter plots
 2. Histograms
 3. Relationship matrix

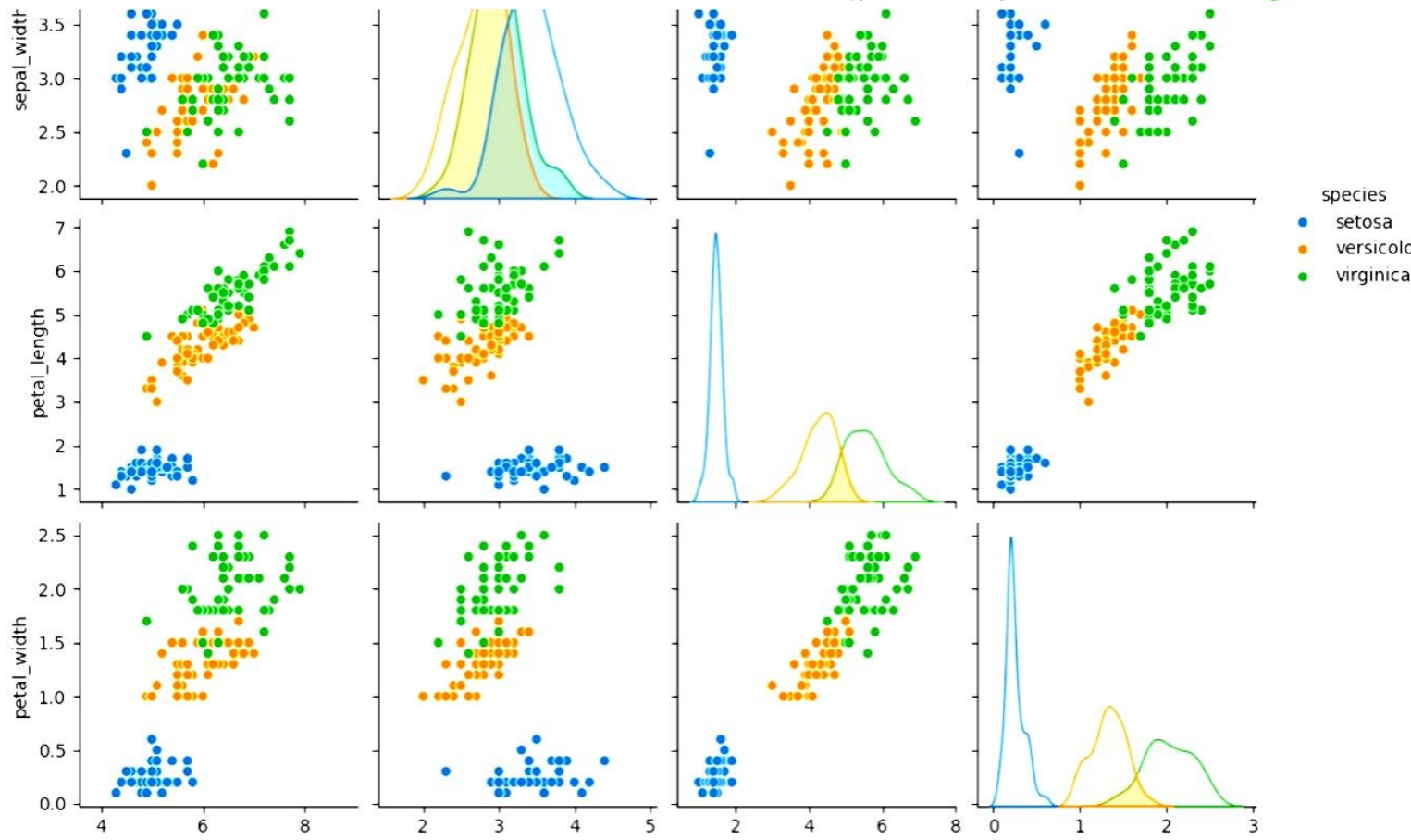
Why use pairplot

- Detect correlation
- Find clusters

```
[17]: #pairplot
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("iris")
sns.pairplot(
    df,
    hue='species' # for blue color lower values so separated
)
plt.show()
```





2. Color-Coded Scatter Plot(3rd Dimension)

- A scatter plot normally shows:

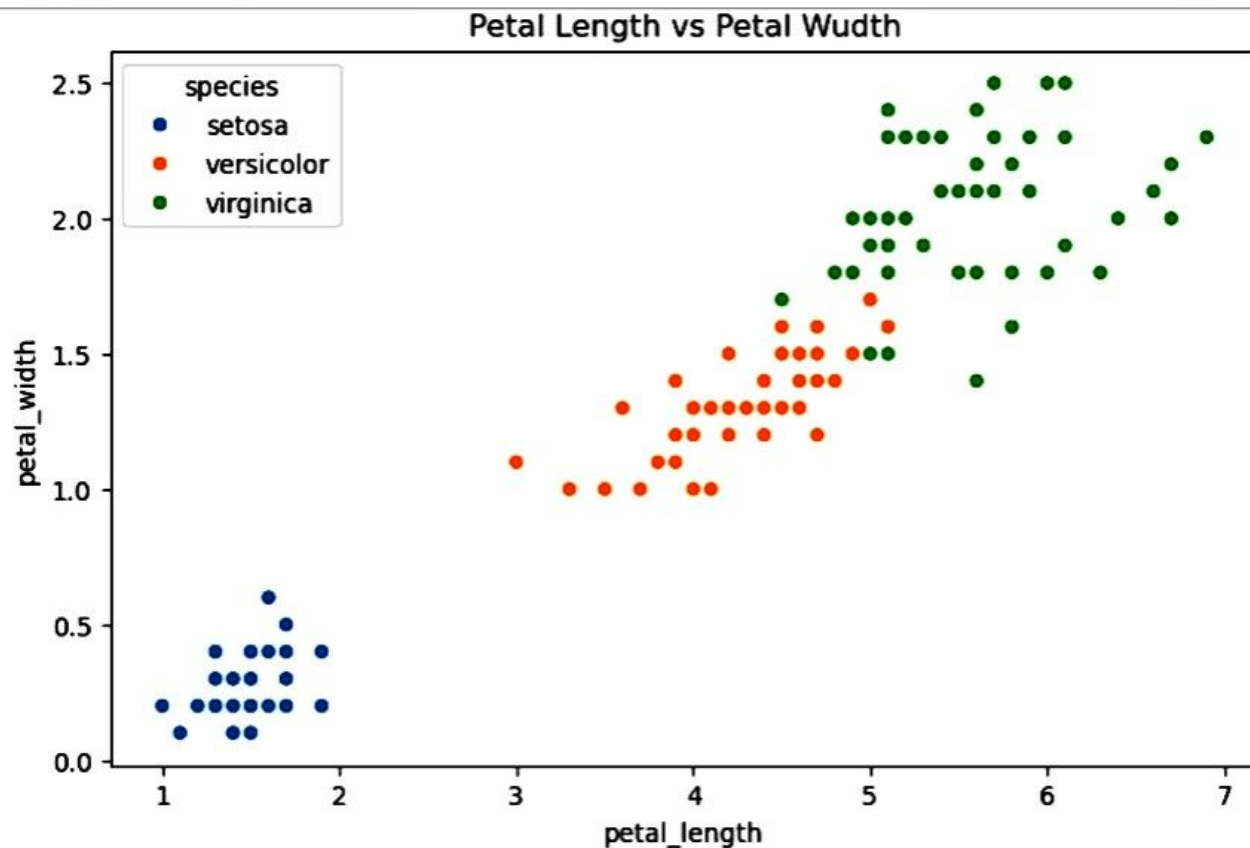
1. X- axis -> Variable 1
2. Y- axis -> Variable 2

A third variable can be represented using colors. Example

1. X -> Petal Length
2. Y -> Petal width
3. Color -> Species

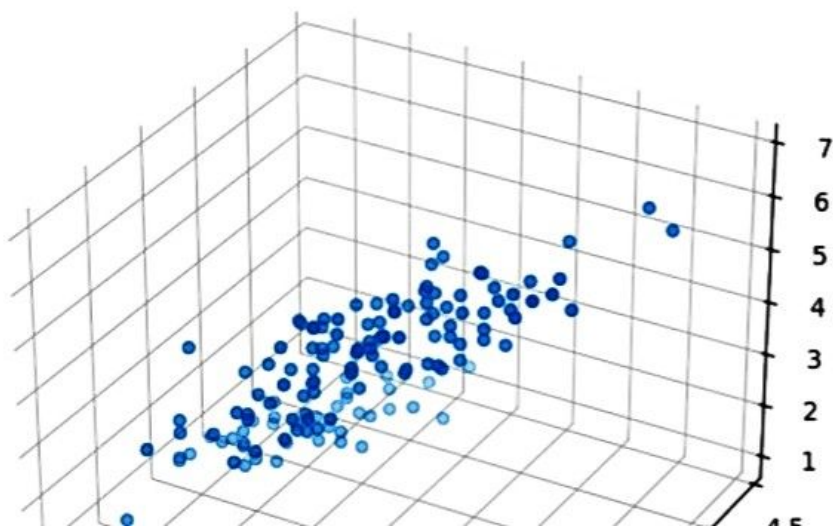
```
[18]: plt.figure(figsize=(8,5))
sns.scatterplot(
    x = 'petal_length',
    y = 'petal_width',
    hue = 'species',
    data = df
)
plt.title("Petal Length vs Petal Wudth")
plt.show()
```

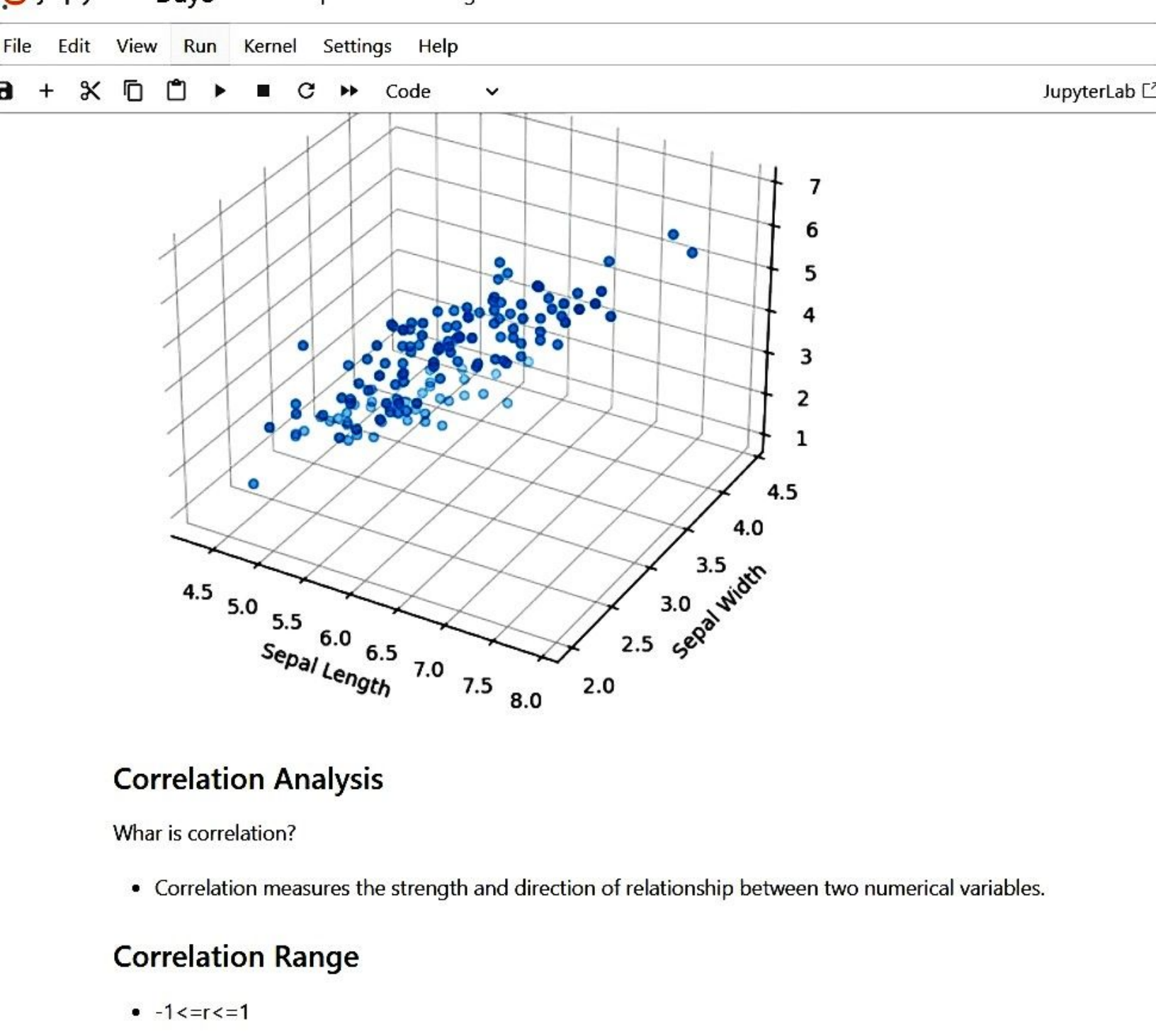




```
[19]: # 3D scatter plot
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111,projection = '3d')
ax.scatter(
    df['sepal_length'],
    df['sepal_width'],
    df['petal_length']
)
ax.set_xlabel('Sepal Length')
ax.set_ylabel('Sepal Width')
ax.set_zlabel('Petal Length')
plt.show()
```





Correlation Analysis

What is correlation?

- Correlation measures the strength and direction of relationship between two numerical variables.

Correlation Range

- $-1 \leq r \leq 1$

```
[20]: # Example Dataset
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("iris")
print(df.head())
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Correlation Matrix

- A correlation matrix shows the correlation between all numerical columns.

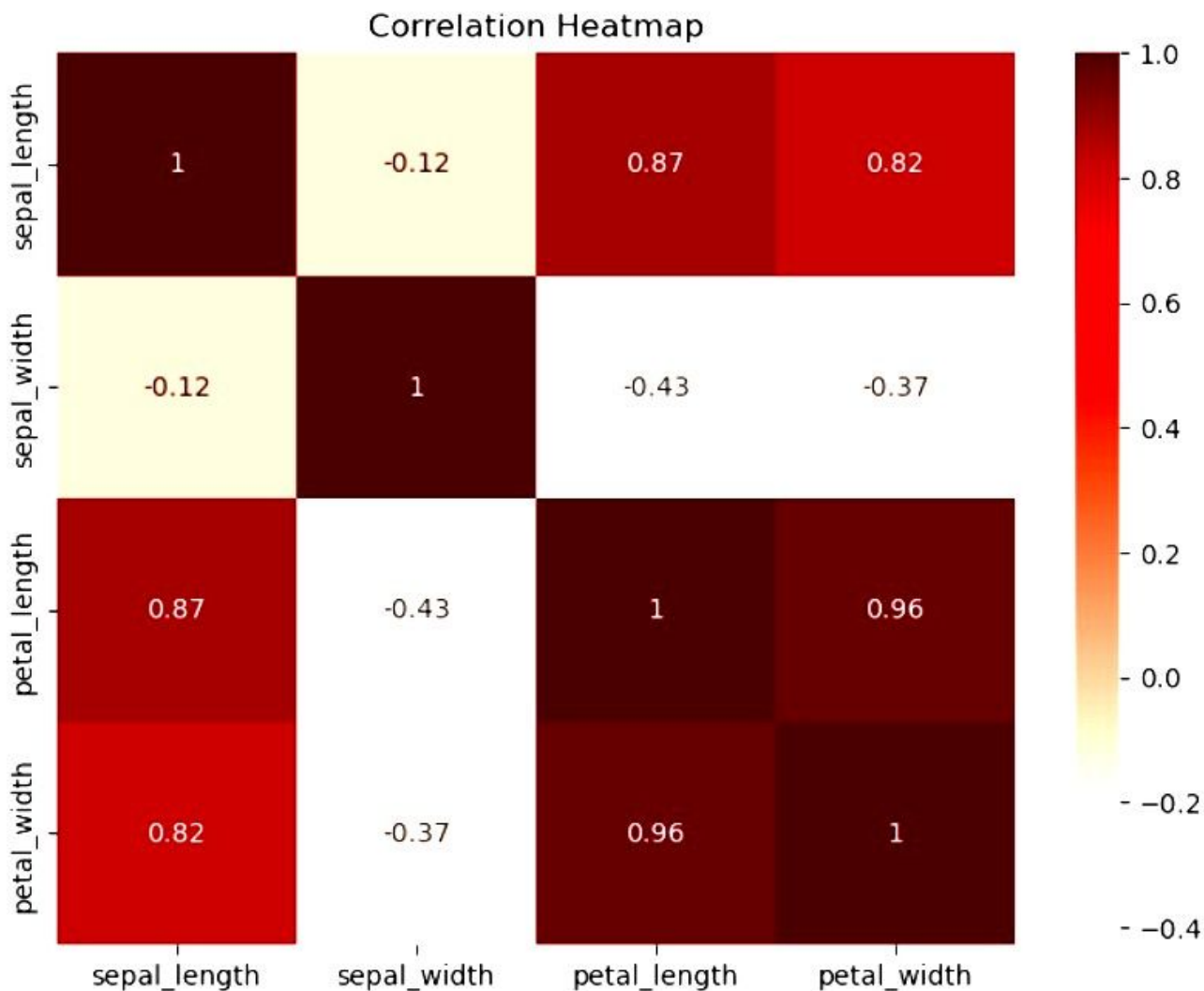
```
[21]: corr_matrix = df.corr(numeric_only=True)
print(corr_matrix)
```

	sepal_length	sepal_width	petal_length	petal_width
sepal_length	1.000000	-0.117570	0.871754	0.817941
sepal_width	-0.117570	1.000000	-0.428440	-0.366126
petal_length	0.871754	-0.428440	1.000000	0.962865
petal_width	0.817941	-0.366126	0.962865	1.000000

```
[ ]: ### Heat map
* Heat map visualizes
```

```
[22]: plt.figure(figsize=(8,6))
sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
```

```
annot=True,
cmap='Reds'
)
plt.title("Correlation Heatmap")
plt.show() # >0.8 means multi collinearity
```



VIF (Variance Inflation Factor) for Multicollinearity Detection

What is Multicollinearity ?

- Multicollinearity occurs when independent variables (features) are highly correlated withh each other.

Example:

- Suppose a house price dataset contains:
 1. House Area(sq.ft)
 2. Number of Rooms 3.Number of Bedrooms
- These features may contain information and be highly correlated
- This creates Multicollinearity.

What is VIF? * Variance Inflation Factor(VIF) measures how much the variance of a feature is inflated because of its correlation with other features.

VIF Interpretation Table VIFValue Interpretation 1 No correlation 1-5 Low correlation 5-10 Moderate Multicollinearity

10 Severe Multicollinearity

```
[23]: from statsmodels.stats.outliers_influence import variance_inflation_factor
X= df.drop("species",axis=1)
vif = pd.DataFrame()

vif["Feature"] = X.columns

vif["VIF"] = [
    variance_inflation_factor(X.values,i)
```

```
vif["VIF"] = [
    variance_inflation_factor(X.values,i)
    for i in range(X.shape[1])
]
print(vif)
```

	Feature	VIF
0	sepal_length	262.969348
1	sepal_width	96.353292
2	petal_length	172.960962
3	petal_width	55.502060

```
[24]: #Remove
      # Suppose we remove petal_length

X_new = X.drop("sepal_length",axis=1)
vif_after = pd.DataFrame()
vif_after["Feature"] = X_new.columns
vif_after["VIF"] = [
    variance_inflation_factor(X_new.values,i)
    for i in range(X_new.shape[1])
]
print(vif_after)
```

	Feature	VIF
0	sepal_width	5.856965
1	petal_length	62.071308
2	petal_width	43.292574

```
[25]: ## Heat Map
plt.figure(figsize=(8,6))
sns.heatmap(
    X_new.corr(),
```



```
annot=True,  
cmap='Greens'  
)  
plt.title("After VIF Treatment")  
plt.show()
```

